



Assignment 2:

Real-World Application of Stack Using Array-Based and Pointer-Based Implementation in C++

**SECJ2013 - DATA STRUCTURE AND
ALGORITHM**

SEMESTER I, SESSION 2024/2025

SECTION 02

Lecturer: Dr. Zuraini Binti Ali Shah

Name	Matric Number
LEE YIN SHEN	A23CS0236
AHMAD ZIYAAD BIN MOHD ABBAS	A23CS0206
ELIJAH SHE YU SHENG	A23CS0073
NURUL ASYIKIN BINTI KHAIRUL ANUAR	A23CS0162

Table of Content

Task 1: Understanding the Concept of Stack	2
Task 2: Application of Stack in a Real-World Scenario	5
Task 3: Implementation in C++	7
Task 4: Comparative Analysis/Findings	14

Task 1: Understanding the Concept of Stack

1.Explain the basic operations of a Stack (Push, Pop, stackTop, IsEmpty, IsFull) with examples (in C++ comments).

```
#include <iostream>
using namespace std;

class Stack
{
    private:
        int * arr;
        int top;
        int capacity;

    public:

        Stack (int size) // constructor of Stack class
        {
            arr = new int [size];
            capacity = size;
            top = -1;
        }

        void push (int x) // push operation adds element at the top
        {
            if (isFull())
            {
                cout << "Stack Overflow" << endl;
                return;
            }

            arr[++top] = x;
            cout << x << " pushed to stack" << endl;
        }

        int pop() // pop operation removes and return the top element
        {
            if (isEmpty())
            {
                cout << "Stack Underflow" << endl;
                return -1;
            }

            cout << arr[top] << " popped from stack" << endl;
            return arr[top--];
        }

        int stackTop() // stackTop operation returns the top element
        {
            if (isEmpty())
            {
                cout << "Stack is empty" << endl;
                return -1;
            }
        }
    };
};
```

```

    }

    return arr[top];

}

bool isEmpty() // isEmpty operation checks if the stack has no elements
{
    return (top == -1);
}

bool isFull() // isFull operation check if the stack reached the capacity or full
{
    return (top == capacity -1);
}

void display()
{
    if (isEmpty())
    {
        cout << "Stack is empty" << endl;
        return;
    }
    cout << "Stack elements: ";
    for (int i = 0; i <= top; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

~Stack()
{
    delete[] arr;
}
};

int main()
{
    Stack s(5);

    cout << "Is stack empty?" << (s.isEmpty() ? "Yes" : "No") << endl;

    s.push(10);
    s.push(20);
    s.push(30);

    s.display();

    cout << "Top element: " << s.stackTop() << endl;
}

```

```
s.pop();  
s.display();  
  
s.push(40);  
  
cout << "Is stack full?" << (s.isFull() ? "Yes" : "No") << endl;  
  
s.display();  
  
system("pause");  
return 0;  
  
}
```

2. Explain the LIFO (Last-In-First-Out) principle and its relevance in real-world scenarios (in reports).

When processing and storing data, the Last-In-First-Out (LIFO) principle states that the item that was added most recently should be removed first. In basic terms, the most recent data are processed or retrieved first, followed by the older data. LIFO has two primary operations which is push that adds an item to the top of the stack and pop which removes the topmost item. These procedures ensure that the stack efficiently applies the Last-In-First-Out principle.

The software undo feature and web browser navigation history are two real-world examples of LIFO in action. LIFO principle gives people a smooth way to edit or change their work by reversing the most recent action taken in programs like word processors and graphic design tools. As well, web browsers use LIFO to store navigation history, with the back button retrieving the most recently seen page, allowing for quick backward navigation through previously viewed pages.

Moreover to ensure correct sequencing and execution order in complicated processes, the LIFO concept is crucial in programming for managing call stacks where the most recently called function is the first to finish and exit.

Task 2: Application of Stack in a Real-World Scenario

1. Identify and choose one real-world application where the concept of Stack is used. Examples include:

- Undo/Redo functionality in text editors.
- Backtracking in algorithms (e.g., maze solving or navigating web browser history).
- **Evaluating mathematical expressions (Infix to Postfix conversion).**
- Postfix Calculator

The Real-world application: **Mathematical Expressions- infix to postfix conversion.**

In mathematics and computer science, arithmetic expressions are often written in **infix notation**, where operators are placed between operands for example $A+B$. However, computers process expressions more efficiently in **postfix notation** (also known as Reverse Polish Notation), where operators follow their operands, $AB+$. The conversion of an infix expression to postfix is a real-world example of the **Stack** data structure in action.

2. Provide a brief explanation of how the Stack data structure supports this application.

Example:

- infix expression: $(15 * 3) / 9 - 20$

- Postfix expression: $15\ 3\ *\ 9\ /\ 20\ -$

Step	Chapter Read(ch)	Stack Content(operator)	Postfix Notation Created
1	$(15 * 3) / 9 - 20$	#	
2	$15 * 3) / 9 - 20$	#(	
3	$* 3) / 9 - 20$	#(	15
4	$3) / 9 - 20$	#(*	15
5	$) / 9 - 20$	#(*	15 3
6	$/ 9 - 20$	#	15 3 *
7	$9 - 20$	#/	15 3 *
8	$- 20$	#/	15 3 * 9
9	20	#-	15 3 * 9/
10		#-	15 3 * 9 / 20
11		#	15 3 * 9 / 20-

Figure 1: Table for infix to postfix expression

Explanation:

Operands: Directly added to the postfix expression.

Operators:

- Pushed onto a stack.
- If the operator at the top of the stack has higher or equal precedence than the current operator, it is popped and added to the postfix expression.

Parentheses:

- A left parenthesis “(” is pushed onto the stack.
- When a right parenthesis “)” is encountered, operators are popped from the stack and added to the postfix expression until a left parenthesis is found.

Stack Operand Used:

- **Push:** Add operators or parentheses to the stack
- **Pop:** Remove operators from the stack when they need to be added to the postfix expression.

The stack data structure plays a crucial role in maintaining the correct order of operations (precedence) when converting infix expressions to postfix. By systematically managing operators and parentheses, the stack ensures that higher-precedence operations are handled before lower-precedence ones, preserving the intended mathematical logic. Additionally, the efficiency of stack operations, such as push, pop, and peek, which are performed in constant $O(1)$ time, contributes to the swift processing of expressions. The inherent Last-In-First-Out (LIFO) property of stacks naturally aligns with the sequence in which operators and operands need to be evaluated, making it an ideal structure for this application.

Task 3: Implementation in C++

1. Implement the chosen real-world application using the Stack data structure in two ways:

- **Array-Based Implementation:** Use a fixed-size array to implement the stack.
- **Pointer-Based Implementation (Linked List):** Use dynamic memory allocation and pointers to implement the stack.

2. Your implementation must include the following operations:

- **Push:** Add an element to the stack.
- **Pop:** Remove an element from the stack.
- **stackTop:** Display the top element of the stack.
- **Display:** Show all elements currently in the stack. o isFull and isEmpty : when is a needed

3. Write a menu-driven program in C++ that allows users to perform the operations interactively.

Array-Based Implementation

```
#include<iostream>
#include<string>
using namespace std ;

const int MAX_SIZE = 100 ;

class Stack{
private:
    char arr[MAX_SIZE] ;
    int top ;

public:
    Stack(){
        top = -1 ;
    }

    void push(char x){
        if(!isFull()){
            arr[++top] = x ;
        }
    }

    char pop(){
        return arr[top--] ;
    }

    char stackTop(){
        return arr[top] ;
    }

    bool isEmpty(){
        return top == -1 ;
    }

    bool isFull(){
```



```

    return top == MAX_SIZE - 1 ;
}

void display(){
    if (isEmpty()){
        cout << "Stack elements: #" << endl ;
        return;
    }

    cout << "Stack elements: #";
    for (int i = 0; i <= top; i++){
        cout << arr[i] ;
    }
    cout << endl ;
}
};

class Converter{
private:
    string input, output ;
    Stack stack ;
    int position ;

    int precedence(char c){
        if(c == '+' || c == '-') return 1 ;
        if(c == '*' || c == '/') return 2 ;
        if(c == '(') return 0 ;
        return 0;
    }

public:
    Converter(string expr){
        input = expr ;
        position = 0 ;
    }

    void next(){
        if(position >= input.length()){
            while(!stack.isEmpty()) {
                char c = stack.pop() ;
                if(c != '(')
                    output += c ;
            }
        }
        else{
            char c = input[position] ;

            if(isdigit(c)){
                // 15 to 15 instead of 1 5
                while(position < input.length() && isdigit(input[position])) {
                    output += input[position] ;
                    position++ ;
                }
            }
            else {

```

```

        position++; // Move position for non-digit characters
        if(isalpha(c)){
            output += c;
        }
        else if(c == '('){
            stack.push(c) ;
        }
        else if(c == ')'){
            while(!stack.isEmpty() && stack.stackTop() != '(') {
                output += stack.pop() ;
            }
            if(!stack.isEmpty() && stack.stackTop() == '(') {
                stack.pop() ;
            }
        }
        else{
            while(!stack.isEmpty() && precedence(c) <= precedence(stack.stackTop())){
                char topChar = stack.pop() ;
                if(topChar != '(')
                    output += topChar ;
            }
            stack.push(c) ;
        }
    }
}

cout << "\nInput: " << input << endl ;
cout << "Current output: " << output << endl ;
stack.display();
}

void displayStack(){
    stack.display() ;
}

void displayTop(){
    cout << "Top element: " << stack.stackTop() << endl ;
}
};

int main(){
    string expr ;
    char choice ;

    cout << " Mathematical Expressions - Infix to Postfix Conversion.";
    cout << "Enter expression: ";
    cin >> expr;

    Converter conv(expr) ;

    while(true){
        cout << "\nN-Next, D-Display, T-Top, Q-Quit: ";
        cin >> choice;

        if(choice == 'N' || choice == 'n') conv.next() ;
    }
}

```

```

        else if(choice == 'D' || choice == 'd') conv.displayStack() ;
        else if(choice == 'T' || choice == 't') conv.displayTop() ;
        else if(choice == 'Q' || choice == 'q') break ;
    }

    return 0;
}

```

Pointer-Based Implementation (Linked List)

```

#include<iostream>
#include<string>
using namespace std ;

const int MAX_SIZE = 100 ;

// Define Node structure for linked list
class Node{
public:
    char data ;
    Node* next ;

    Node(char value){
        data = value ;
        next = nullptr ;
    }
};

class Stack{
private:
    Node* top ; // Changed from array to pointer

public:
    Stack(){
        top = nullptr;
    }

    void push(char x){
        Node* newNode = new Node(x) ;
        newNode->next = top ;
        top = newNode ;
    }

    char pop(){
        if (isEmpty()) return '\0' ;
        char value = top->data ;
        Node* temp = top ;
        top = top->next ;
        delete temp ;
        return value ;
    }

    char stackTop(){

```

```

        if(isEmpty()) return '\0' ;
        return top->data ;
    }

    bool isEmpty(){
        return top == nullptr ;
    }

    void display(){
        if (isEmpty()){
            cout << "Stack elements: #" << endl ;
            return ;
        }

        cout << "Stack elements: #" ;
        Node* current = top ;
        while (current != nullptr){
            cout << current->data ;
            current = current->next ;
        }
        cout << endl ;
    }

    // Add destructor to free memory
    ~Stack(){
        while (!isEmpty()) {
            pop();
        }
    }
};

class Converter{
private:
    string input, output ;
    Stack stack ;
    int position ;

    int precedence(char c){
        if(c == '+' || c == '-') return 1 ;
        if(c == '*' || c == '/') return 2 ;
        if(c == '(') return 0 ;
        return 0 ;
    }

public:
    Converter(string expr){
        input = expr ;
        position = 0 ;
    }

    void next(){
        if(position >= input.length()){
            while(!stack.isEmpty()){
                char c = stack.pop() ;
                if(c != '(')

```

```

        output += c ;
    }
}
else{
    char c = input[position] ;

    if(isdigit(c)) {
        // 15 to 15 instead of 1 5
        while(position < input.length() && isdigit(input[position])){
            output += input[position] ;
            position++ ;
        }
    }
    else{
        position++; // Move position for non-digit characters
        if(isalpha(c)){
            output += c ;
        }
        else if(c == '('){
            stack.push(c) ;
        }
        else if(c == ')'){
            while(!stack.isEmpty() && stack.stackTop() != '('){
                output += stack.pop() ;
            }
            if(!stack.isEmpty() && stack.stackTop() == '('){
                stack.pop() ;
            }
        }
        else{
            while(!stack.isEmpty() && precedence(c) <= precedence(stack.stackTop())){
                char topChar = stack.pop() ;
                if(topChar != '(')
                    output += topChar ;
            }
            stack.push(c) ;
        }
    }
}

cout << "\nInput: " << input << endl ;
cout << "Current output: " << output << endl ;
stack.display() ;
}

void displayStack(){
    stack.display() ;
}

void displayTop(){
    cout << "Top element: " << stack.stackTop() << endl ;
}
};

int main(){

```

```
string expr ;
char choice ;

cout << " Mathematical Expressions - Infix to Postfix Conversion.";
cout << "Enter expression: " ;
cin >> expr;

Converter conv(expr) ;

while(true){
    cout << "\nN-Next, D-Display, T-Top, Q-Quit: " ;
    cin >> choice;

    if(choice == 'N' || choice == 'n') conv.next() ;
    else if(choice == 'D' || choice == 'd') conv.displayStack() ;
    else if(choice == 'T' || choice == 't') conv.displayTop() ;
    else if(choice == 'Q' || choice == 'q') break ;
}

return 0 ;
}
```

Task 4: Comparative Analysis/Findings

1. Compare the Array-Based and Pointer-Based implementations in terms of:

- **Memory usage.**
- **Ease of implementation.**
- **Scalability.**

2. Write your findings in a summary report (200-300 words).

Contiguous memory blocks are usually needed for array-based solutions, which can result in inefficient memory use, particularly if the array needs to be resized or is poorly populated. Resizing an array frequently necessitates copying components and creating a new block of memory, both of which can be expensive. On the other hand, memory is dynamically allocated for every element in pointer-based solutions, like linked lists. This reduces wasted space and eliminates the need for contiguous memory. But because pointers need more memory to be stored, pointer-based structures have overhead, which can pile up for big datasets.

Generally speaking, array-based systems are simpler to set up and troubleshoot. They make many processes simpler by providing direct access to elements through indexes. On the other hand, because pointers must be deliberately managed, pointer-based implementations are more complicated. In order to prevent memory leaks or dangling references, pointers must be handled carefully when performing operations like insertion, deletion, and traversal. Pointer-based structures are more prone to errors because of their complexity, particularly for novices.

Implementations based on arrays scale effectively for applications with fixed or predictable sizes. However, the complexity of resizing operations and the need for contiguous memory limit their scalability in dynamic circumstances. Because pointer-based solutions may readily expand or contract by allocating or freeing memory for individual pieces, they perform exceptionally well in dynamic contexts. They are therefore better suited for situations in which the dataset's size fluctuates regularly or is uncertain.

Implementations based on arrays and pointers each have unique benefits and drawbacks. For static datasets, array-based structures are easier to create, enable quick access using indices, and use less memory. Their scaling restrictions and dependence on contiguous memory, however, limit their scalability. Linked lists and other pointer-based solutions offer dynamic memory management, enabling smooth scalability for datasets of varying sizes. Despite their adaptability, they are more difficult to create and maintain due to the intricacy of pointer manipulation and require more memory because of pointer overhead. The particular application requirements, such as developer experience, performance limitations, and dataset size predictability, will determine which option is best.